🚀 MERN Stack Developer সম্পূর্ণ রোডম্যাপ (বাংলায়)

📋 সূচিপত্র
text
Phase 1: ভিত্তি (Foundation)
Phase 2: Frontend Development
Phase 3: Backend Development
Phase 4: Database
Phase 5: Full Stack Integration
Phase 6: Advanced Topics
Phase 7: DevOps & Deployment
Phase 8: Professional Skills

🟢 Phase 1: ভিত্তি (Foundation) — সময়: ৪-৬ সপ্তাহ

1.1 ইন্টারনেট ও ওয়েব কিভাবে কাজ করে
text
বিষয়সমূহ:
├── ইন্টারনেট কি এবং কিভাবে কাজ করে
├── HTTP/HTTPS কি
├── DNS কিভাবে কাজ করে
├── Domain ও Hosting কি
├── Browser কিভাবে কাজ করে
├── Client-Server Architecture
└── API কি
ব্যাখ্যা:
ওয়েব ডেভেলপমেন্ট শুরু করার আগে আপনাকে বুঝতে হবে ইন্টারনেট কিভাবে কাজ করে। যখন আপনি ব্রাউজারে কোনো URL লেখেন, তখন ব্রাউজার DNS সার্ভারে গিয়ে সেই ডোমেইনের IP Address খুঁজে বের করে, তারপর সেই সার্ভারে HTTP Request পাঠায় এবং সার্ভার Response হিসেবে HTML, CSS, JavaScript ফাইল ফেরত পাঠায়। Client হলো ব্রাউজার, Server হলো যেখানে আপনার ওয়েবসাইট হোস্ট করা আছে। API হলো দুটি সফটওয়্যারের মধ্যে যোগাযোগের মাধ্যম।

1.2 HTML (HyperText Markup Language)
text
বিষয়সমূহ:
├── HTML Basics
│ ├── HTML কি এবং কেন দরকার
│ ├── HTML Document Structure (<!DOCTYPE>, <html>, <head>, <body>)
│ ├── Headings (<h1> থেকে <h6>)
│ ├── Paragraphs (<p>)
│ ├── Links (<a>)
│ ├── Images (<img>)
│ ├── Lists (Ordered <ol>, Unordered <ul>)
│ └── Comments

```
│
├── HTML Intermediate
│   ├── Tables (<table>, <tr>, <td>, <th>)
│   ├── Forms (<form>, <input>, <textarea>, <select>)
│   ├── Input Types (text, email, password, number, date, file, radio, checkbox)
│   ├── Form Validation (required, pattern, min, max)
│   ├── Div ও Span
│   ├── Block vs Inline Elements
│   └── HTML Entities
│
├── HTML5 Semantic Elements
│   ├── <header>, <footer>
│   ├── <nav>, <main>
│   ├── <section>, <article>
│   ├── <aside>, <figure>, <figcaption>
│   └── কেন Semantic HTML গুরুত্বপূর্ণ (SEO ও Accessibility)
│
├── HTML5 Advanced
│   ├── Audio ও Video Embed
│   ├── Canvas
│   ├── SVG
│   ├── Data Attributes (data-*)
│   ├── Meta Tags (viewport, description, keywords)
│   └── Open Graph Tags
│
└── Accessibility (a11y)
    ├── Alt Text for Images
    ├── ARIA Labels
    ├── Semantic Structure
    └── Keyboard Navigation
```

ব্যাখ্যা:
HTML হলো ওয়েব পেজের কঙ্কাল বা হাড্ডি। এটি দিয়ে ওয়েব পেজের গঠন (Structure) তৈরি করা হয়। প্রতিটি HTML Element একটি Tag দিয়ে শুরু হয় এবং শেষ হয়। HTML5 এ Semantic Elements এসেছে যা ব্রাউজার এবং সার্চ ইঞ্জিনকে বোঝায় পেজের কোন অংশে কি আছে। যেমন <nav> মানে Navigation, <header> মানে হেডার অংশ। Forms ব্যবহার করে ইউজারের কাছ থেকে ডাটা নেওয়া হয়। Accessibility মানে প্রতিবন্ধী ব্যক্তিরাও যেন ওয়েবসাইট ব্যবহার করতে পারে।

1.3 CSS (Cascading Style Sheets)
text
বিষয়সমূহ:

```
├── CSS Basics
│   ├── CSS কি এবং কিভাবে কাজ করে
│   ├── CSS যুক্ত করার ৩টি পদ্ধতি (Inline, Internal, External)
│   ├── Selectors
```

```
│   │   ├── Element Selector (h1, p, div)
│   │   ├── Class Selector (.className)
│   │   ├── ID Selector (#idName)
│   │   ├── Universal Selector (*)
│   │   ├── Grouping Selector (h1, h2, p)
│   │   ├── Descendant Selector (div p)
│   │   ├── Child Selector (div > p)
│   │   ├── Attribute Selector (input[type="text"])
│   │   └── Pseudo-classes (:hover, :focus, :nth-child)
│   │
│   ├── Colors (hex, rgb, rgba, hsl)
│   ├── Backgrounds (color, image, gradient)
│   ├── Typography (font-family, font-size, font-weight, line-height)
│   ├── Text Properties (text-align, text-decoration, text-transform)
│   └── Units (px, em, rem, %, vh, vw)
│
├── Box Model
│   ├── Content
│   ├── Padding
│   ├── Border
│   ├── Margin
│   ├── box-sizing: border-box
│   └── Outline
│
├── CSS Layout
│   ├── Display (block, inline, inline-block, none, flex, grid)
│   ├── Position (static, relative, absolute, fixed, sticky)
│   ├── Float ও Clear (পুরনো পদ্ধতি)
│   ├── Overflow (visible, hidden, scroll, auto)
│   └── Z-index
│
├── Flexbox (অত্যন্ত গুরুত্বপূর্ণ)
│   ├── display: flex
│   ├── flex-direction (row, column, row-reverse, column-reverse)
│   ├── justify-content (center, space-between, space-around, flex-start, flex-end)
│   ├── align-items (center, stretch, flex-start, flex-end)
│   ├── flex-wrap
│   ├── gap
│   ├── flex-grow, flex-shrink, flex-basis
│   ├── align-self
│   └── order
│
├── CSS Grid (অত্যন্ত গুরুত্বপূর্ণ)
│   ├── display: grid
```

```
│   ├── grid-template-columns
│   ├── grid-template-rows
│   ├── grid-gap / gap
│   ├── grid-column, grid-row
│   ├── grid-area
│   ├── grid-template-areas
│   ├── fr unit
│   ├── repeat(), minmax()
│   ├── auto-fit, auto-fill
│   └── justify-items, align-items, place-items
│
├── Responsive Design
│   ├── Media Queries (@media)
│   ├── Mobile-First Approach
│   ├── Breakpoints (576px, 768px, 992px, 1200px)
│   ├── Responsive Images
│   ├── Viewport Meta Tag
│   └── Responsive Typography
│
├── CSS Advanced
│   ├── Transitions (transition-property, duration, timing-function)
│   ├── Animations (@keyframes, animation)
│   ├── Transforms (translate, rotate, scale, skew)
│   ├── CSS Variables (Custom Properties)
│   │   └── --primary-color: #007bff; → var(--primary-color)
│   ├── Pseudo-elements (::before, ::after)
│   ├── Filters (blur, brightness, contrast)
│   ├── Clip-path
│   ├── CSS Functions (calc(), min(), max(), clamp())
│   └── Object-fit, Object-position
│
└── CSS Frameworks ও Tools
    ├── Tailwind CSS (সবচেয়ে জনপ্রিয় - অবশ্যই শিখুন)
    │   ├── Utility-First Approach
    │   ├── Responsive Design with Tailwind
    │   ├── Custom Configuration
    │   ├── Dark Mode
    │   └── Tailwind Components
    │
    ├── Bootstrap (ঐচ্ছিক)
    ├── CSS Modules
    ├── Styled Components (React এর জন্য)
    ├── SASS/SCSS (CSS Preprocessor)
    │   ├── Variables
```

```
│   ├── Nesting
│   ├── Mixins
│   ├── Partials ও Import
│   └── Functions
└── PostCSS
```

ব্যাখ্যা:
CSS দিয়ে HTML এর সাজসজ্জা করা হয়। Box Model বুঝা খুবই জরুরি - প্রতিটি HTML Element একটি বক্স, যার Content, Padding (ভিতরের ফাঁকা জায়গা), Border (সীমানা) এবং Margin (বাইরের ফাঁকা জায়গা) আছে। Flexbox হলো এক মাত্রার (1D) লেআউট সিস্টেম - এক সারিতে বা এক কলামে Element সাজাতে ব্যবহার করা হয়। CSS Grid হলো দুই মাত্রার (2D) লেআউট সিস্টেম - সারি ও কলাম দুটোতেই একসাথে কাজ করা যায়। Responsive Design মানে ওয়েবসাইট যেকোনো স্ক্রিন সাইজে সুন্দর দেখাবে। Tailwind CSS একটি Utility-First Framework যেটা আজকের দিনে সবচেয়ে জনপ্রিয়।

1.4 JavaScript (সবচেয়ে গুরুত্বপূর্ণ)
text
বিষয়সমূহ:

```
├── JS Basics
│   ├── JavaScript কি ও কেন দরকার
│   ├── Script Tag ও External JS File
│   ├── Console.log()
│   ├── Variables (var, let, const)
│   │   ├── var → Function Scoped, Hoisted, Re-declarable
│   │   ├── let → Block Scoped, Not Hoisted (TDZ), Re-assignable
│   │   └── const → Block Scoped, Not Hoisted (TDZ), Not Re-assignable
│   ├── Data Types
│   │   ├── Primitive: String, Number, Boolean, Null, Undefined, Symbol, BigInt
│   │   └── Non-Primitive: Object, Array, Function
│   ├── Type Conversion ও Type Coercion
│   ├── Operators
│   │   ├── Arithmetic (+, -, *, /, %, **)
│   │   ├── Assignment (=, +=, -=, *=, /=)
│   │   ├── Comparison (==, ===, !=, !==, >, <, >=, <=)
│   │   ├── Logical (&&, ||, !)
│   │   ├── Ternary (condition ? true : false)
│   │   ├── Nullish Coalescing (??)
│   │   ├── Optional Chaining (?.)
│   │   └── Spread (...) ও Rest (...)
│   ├── Template Literals (`Hello ${name}`)
│   └── typeof operator
│
├── Control Flow
│   ├── if, else if, else
│   ├── switch-case
│   ├── for loop
```

```
│   ├── while loop
│   ├── do-while loop
│   ├── for...of (Array/String iterate)
│   ├── for...in (Object iterate)
│   ├── break ও continue
│   └── Labeled Statements
│
├── Functions (অত্যন্ত গুরুত্বপূর্ণ)
│   ├── Function Declaration
│   │   └── function greet(name) { return `Hello ${name}`; }
│   ├── Function Expression
│   │   └── const greet = function(name) { return `Hello ${name}`; }
│   ├── Arrow Function (ES6)
│   │   └── const greet = (name) => `Hello ${name}`;
│   ├── Default Parameters
│   ├── Rest Parameters (...args)
│   ├── Callback Function
│   ├── Higher-Order Function (HOF)
│   ├── IIFE (Immediately Invoked Function Expression)
│   ├── Pure Function
│   ├── Closure (অত্যন্ত গুরুত্বপূর্ণ)
│   │   └── ফাংশন তার বাইরের স্কোপের ভ্যারিয়েবল মনে রাখে
│   ├── Recursion
│   └── Function Currying
│
├── Arrays (অত্যন্ত গুরুত্বপূর্ণ)
│   ├── Array তৈরি ও Access
│   ├── Array Methods
│   │   ├── Mutating Methods
│   │   │   ├── push() → শেষে যোগ
│   │   │   ├── pop() → শেষ থেকে বাদ
│   │   │   ├── unshift() → শুরুতে যোগ
│   │   │   ├── shift() → শুরু থেকে বাদ
│   │   │   ├── splice() → যেকোনো জায়গায় যোগ/বাদ
│   │   │   ├── sort() → সাজানো
│   │   │   ├── reverse() → উল্টানো
│   │   │   └── fill()
│   │   │
│   │   ├── Non-Mutating Methods (নতুন Array রিটার্ন করে)
│   │   │   ├── map() → প্রতিটি element এ কিছু করে নতুন array
│   │   │   ├── filter() → শর্ত অনুযায়ী ফিল্টার
│   │   │   ├── reduce() → সব element কে একটি মানে নিয়ে আসে
│   │   │   ├── find() → প্রথম মিল খুঁজে
│   │   │   ├── findIndex() → প্রথম মিলের index
```

```
│   │   │   ├── some() → কমপক্ষে একটি মিললে true
│   │   │   ├── every() → সবগুলো মিললে true
│   │   │   ├── includes() → আছে কিনা check
│   │   │   ├── indexOf()
│   │   │   ├── slice() → অংশ কেটে নেওয়া
│   │   │   ├── concat() → জোড়া লাগানো
│   │   │   ├── flat() → নেস্টেড array সমান করা
│   │   │   ├── flatMap()
│   │   │   └── Array.from()
│   │   │
│   │   └── forEach() → লুপ করে (কিছু return করে না)
│   │
│   ├── Array Destructuring
│   │   └── const [a, b, ...rest] = [1, 2, 3, 4, 5];
│   └── Spread Operator with Arrays
│       └── const newArr = [...arr1, ...arr2];
│
├── Objects (অত্যন্ত গুরুত্বপূর্ণ)
│   ├── Object তৈরি ও Access (dot ও bracket notation)
│   ├── Object Methods
│   │   ├── Object.keys() → সব key এর array
│   │   ├── Object.values() → সব value এর array
│   │   ├── Object.entries() → [key, value] pair এর array
│   │   ├── Object.assign() → copy/merge
│   │   ├── Object.freeze() → পরিবর্তন বন্ধ
│   │   ├── Object.seal() → নতুন property যোগ বন্ধ
│   │   └── hasOwnProperty()
│   ├── Object Destructuring
│   │   └── const { name, age, ...rest } = person;
│   ├── Spread Operator with Objects
│   │   └── const newObj = { ...obj1, ...obj2 };
│   ├── Computed Property Names
│   │   └── const obj = { [dynamicKey]: value };
│   ├── Shorthand Properties
│   ├── this keyword (অত্যন্ত গুরুত্বপূর্ণ)
│   ├── JSON (JavaScript Object Notation)
│   │   ├── JSON.stringify() → Object to JSON String
│   │   └── JSON.parse() → JSON String to Object
│   └── Optional Chaining (?.)
│
├── Strings
│   ├── String Methods
│   │   ├── length, charAt(), charCodeAt()
│   │   ├── toUpperCase(), toLowerCase()
```

```
│   │   ├── trim(), trimStart(), trimEnd()
│   │   ├── split() → String to Array
│   │   ├── join() → Array to String
│   │   ├── slice(), substring()
│   │   ├── replace(), replaceAll()
│   │   ├── includes(), startsWith(), endsWith()
│   │   ├── indexOf(), lastIndexOf()
│   │   ├── search(), match()
│   │   ├── repeat(), padStart(), padEnd()
│   │   └── at()
│   └── Regular Expressions (Regex) - বেসিক
│
├── ES6+ Features (Modern JavaScript - অবশ্যই শিখুন)
│   ├── let ও const
│   ├── Arrow Functions
│   ├── Template Literals
│   ├── Destructuring (Array ও Object)
│   ├── Spread ও Rest Operator
│   ├── Default Parameters
│   ├── Enhanced Object Literals
│   ├── for...of Loop
│   ├── Symbol
│   ├── Map ও Set
│   │   ├── Map → key-value pair (যেকোনো type key হতে পারে)
│   │   └── Set → unique values only
│   ├── WeakMap ও WeakSet
│   ├── Iterators ও Generators
│   ├── Optional Chaining (?.)
│   ├── Nullish Coalescing (??)
│   ├── Logical Assignment (&&=, ||=, ??=)
│   └── Array.at(), Object.hasOwn()
│
├── Asynchronous JavaScript (অত্যন্ত গুরুত্বপূর্ণ)
│   ├── কেন Async দরকার
│   │   └── JavaScript Single-Threaded, তাই সময়সাপেক্ষ কাজ
│   │       block না করে background এ করতে হয়
│   ├── setTimeout() ও setInterval()
│   ├── Callback Functions
│   ├── Callback Hell (সমস্যা)
│   ├── Promises (সমাধান)
│   │   ├── Promise কি → ভবিষ্যতে একটি মান পাওয়ার প্রতিশ্রুতি
│   │   ├── States: Pending, Fulfilled, Rejected
│   │   ├── .then() → সফল হলে
│   │   ├── .catch() → ব্যর্থ হলে
```

```
│   │   ├── .finally() → যেকোনো অবস্থায়
│   │   ├── Promise.all() → সব Promise একসাথে
│   │   ├── Promise.allSettled()
│   │   ├── Promise.race() → প্রথমটি শেষ হলেই
│   │   └── Promise.any()
│   │
│   ├── Async/Await (সবচেয়ে সুন্দর পদ্ধতি)
│   │   ├── async function → Promise return করে
│   │   ├── await → Promise resolve হওয়ার জন্য অপেক্ষা
│   │   └── try-catch দিয়ে Error Handling
│   │
│   ├── Fetch API
│   │   ├── GET Request
│   │   ├── POST Request
│   │   ├── PUT/PATCH Request
│   │   ├── DELETE Request
│   │   ├── Headers ও Body
│   │   └── Response handling (.json(), .text())
│   │
│   └── Event Loop (অত্যন্ত গুরুত্বপূর্ণ - Interview এ আসে)
│       ├── Call Stack
│       ├── Web APIs
│       ├── Callback Queue (Task Queue)
│       ├── Microtask Queue (Promise Queue)
│       └── Event Loop কিভাবে কাজ করে
│
├── DOM Manipulation
│   ├── DOM কি → HTML Document এর Object Representation
│   ├── Selecting Elements
│   │   ├── document.getElementById()
│   │   ├── document.getElementsByClassName()
│   │   ├── document.getElementsByTagName()
│   │   ├── document.querySelector() (সবচেয়ে বেশি ব্যবহৃত)
│   │   └── document.querySelectorAll()
│   ├── Modifying Elements
│   │   ├── textContent, innerText, innerHTML
│   │   ├── setAttribute(), getAttribute(), removeAttribute()
│   │   ├── classList.add(), classList.remove(), classList.toggle()
│   │   └── style property
│   ├── Creating ও Removing Elements
│   │   ├── document.createElement()
│   │   ├── appendChild(), prepend()
│   │   ├── insertBefore(), insertAdjacentHTML()
│   │   └── removeChild(), remove()
```

```
│   ├── Event Handling
│   │   ├── addEventListener()
│   │   ├── Event Types (click, submit, input, change, keydown, keyup,
│   │   │   mouseover, mouseout, scroll, load, DOMContentLoaded)
│   │   ├── Event Object (e.target, e.preventDefault(), e.stopPropagation())
│   │   ├── Event Bubbling ও Capturing
│   │   └── Event Delegation
│   └── DOM Traversal
│       ├── parentElement, parentNode
│       ├── children, childNodes
│       ├── firstElementChild, lastElementChild
│       ├── nextElementSibling, previousElementSibling
│       └── closest()
│
├── Browser APIs
│   ├── Local Storage
│   │   ├── localStorage.setItem(key, value)
│   │   ├── localStorage.getItem(key)
│   │   ├── localStorage.removeItem(key)
│   │   └── localStorage.clear()
│   ├── Session Storage (ব্রাউজার বন্ধ করলে মুছে যায়)
│   ├── Cookies
│   ├── Geolocation API
│   ├── History API
│   ├── Navigator API
│   └── Intersection Observer API
│
├── OOP (Object-Oriented Programming)
│   ├── Class ও Constructor
│   ├── Methods
│   ├── Inheritance (extends)
│   ├── super keyword
│   ├── Encapsulation (Private fields #)
│   ├── Polymorphism
│   ├── Static Methods
│   ├── Getters ও Setters
│   ├── Prototype ও Prototypal Inheritance
│   ├── instanceof
│   └── new keyword কিভাবে কাজ করে
│
├── Error Handling
│   ├── try-catch-finally
│   ├── throw new Error()
│   ├── Custom Errors
```

```
│   ├── Error Types (TypeError, ReferenceError, SyntaxError, RangeError)
│   └── Debugging (console, breakpoints, DevTools)
│
├── Modules
│   ├── ES Modules (import/export)
│   │   ├── Named Export/Import
│   │   ├── Default Export/Import
│   │   └── Dynamic Import (import())
│   └── CommonJS (require/module.exports) → Node.js এ ব্যবহৃত
│
└── Advanced JavaScript Concepts
    ├── Execution Context
    ├── Scope Chain
    ├── Hoisting
    ├── Temporal Dead Zone (TDZ)
    ├── this binding (implicit, explicit, new, default)
    ├── call(), apply(), bind()
    ├── Closure
    ├── Currying
    ├── Memoization
    ├── Debouncing ও Throttling
    ├── Shallow Copy vs Deep Copy
    ├── Immutability
    ├── Garbage Collection
    ├── Memory Leaks
    ├── Web Workers
    ├── Service Workers
    └── Proxy ও Reflect
```

ব্যাখ্যা:
JavaScript হলো MERN Stack এর মূল ভিত্তি - Frontend (React), Backend (Node.js/Express), সবখানেই JavaScript। var ফাংশন স্কোপড, let ও const ব্লক স্কোপড। Closure মানে একটি ফাংশন তার তৈরির সময়ের স্কোপের ভ্যারিয়েবলগুলো মনে রাখে। Async/Await দিয়ে API call করা হয় - await লিখলে Promise resolve না হওয়া পর্যন্ত অপেক্ষা করে। Event Loop বুঝা জরুরি - JavaScript single-threaded, তাই async কাজগুলো Web API তে চলে যায়, শেষ হলে Callback/Microtask Queue তে আসে, Call Stack খালি হলে Event Loop সেখান থেকে নিয়ে আসে।

1.5 Version Control (Git & GitHub)
text
বিষয়সমূহ:

```
├── Git Basics
│   ├── Git কি ও কেন দরকার
│   ├── git init → নতুন repository তৈরি
│   ├── git add . → সব ফাইল staging area তে
│   ├── git commit -m "message" → পরিবর্তন সংরক্ষণ
```

```
│   ├── git status → বর্তমান অবস্থা দেখা
│   ├── git log → commit history দেখা
│   ├── git diff → পরিবর্তন দেখা
│   └── .gitignore ফাইল
│
├── Branching
│   ├── git branch → সব branch দেখা
│   ├── git branch feature-name → নতুন branch তৈরি
│   ├── git checkout branch-name → branch পরিবর্তন
│   ├── git checkout -b branch-name → তৈরি + পরিবর্তন
│   ├── git switch branch-name (নতুন)
│   ├── git merge branch-name → branch মিলানো
│   ├── git rebase
│   └── Merge Conflicts সমাধান
│
├── Remote Repository (GitHub)
│   ├── git remote add origin URL
│   ├── git push origin main
│   ├── git pull origin main
│   ├── git clone URL
│   ├── git fetch
│   ├── Fork ও Pull Request
│   └── GitHub Issues ও Projects
│
├── Advanced Git
│   ├── git stash → অসম্পূর্ণ কাজ সাময়িক সংরক্ষণ
│   ├── git reset (soft, mixed, hard)
│   ├── git revert
│   ├── git cherry-pick
│   ├── git tag
│   └── git bisect
│
└── Git Workflow
    ├── Gitflow
    ├── Feature Branch Workflow
    └── Trunk-Based Development
```

ব্যাখ্যা:
Git হলো Version Control System - আপনার কোডের প্রতিটি পরিবর্তনের ইতিহাস রাখে। কোনো ভুল করলে আগের অবস্থায় ফিরে যেতে পারবেন। Branch মানে মূল কোড থেকে আলাদা একটি শাখায় কাজ করা - কাজ শেষ হলে Merge করে মূল কোডে যোগ করা যায়। GitHub হলো Git repository রাখার অনলাইন প্ল্যাটফর্ম এবং অন্যদের সাথে সহযোগিতা করার জায়গা। Pull Request মানে আপনার পরিবর্তন অন্যদের Review করতে দেওয়া।

🔵 Phase 2: Frontend Development (React.js) — সময়: ৮-১২ সপ্তাহ

2.1 React.js Fundamentals
text
বিষয়সমূহ:

```
├── React পরিচিতি
│   ├── React কি → Facebook এর তৈরি JavaScript Library
│   ├── কেন React → Component-Based, Virtual DOM, Reusable
│   ├── SPA (Single Page Application) কি
│   ├── Virtual DOM কিভাবে কাজ করে
│   │   └── Real DOM পরিবর্তন ব্যয়বহুল, তাই React প্রথমে
│   │       Virtual DOM এ পরিবর্তন করে, তারপর শুধু
│   │       পরিবর্তিত অংশটুকু Real DOM এ আপডেট করে (Diffing)
│   ├── React vs Angular vs Vue তুলনা
│   └── Create React App vs Vite
│       └── Vite ব্যবহার করুন (দ্রুততর)
│           └── npm create vite@latest my-app -- --template react
│
├── JSX (JavaScript XML)
│   ├── JSX কি → JavaScript এ HTML লেখার সুবিধা
│   ├── JSX Rules
│   │   ├── একটি Parent Element থাকতে হবে (<> </> Fragment ব্যবহার করুন)
│   │   ├── class → className
│   │   ├── for → htmlFor
│   │   ├── Self-closing tags (<img />, <br />)
│   │   └── JavaScript Expression {} দিয়ে
│   ├── Conditional Rendering
│   │   ├── if-else (JSX এর বাইরে)
│   │   ├── Ternary Operator (condition ? <A /> : <B />)
│   │   └── Short Circuit (condition && <A />)
│   └── List Rendering
│       └── array.map((item) => <li key={item.id}>{item.name}</li>)
│           └── key prop অবশ্যই দিতে হবে (unique identifier)
│
├── Components
│   ├── Functional Components (এটাই শিখুন, Class Component পুরনো)
│   │   └── function App() { return <h1>Hello</h1>; }
│   │   └── const App = () => { return <h1>Hello</h1>; }
│   ├── Component Naming Convention → PascalCase (MyComponent)
│   ├── Component Composition → ছোট ছোট component দিয়ে বড় UI তৈরি
│   ├── Props (Properties)
│   │   ├── Parent থেকে Child এ Data পাঠানো
│   │   ├── Props হলো Read-Only (Immutable)
│   │   ├── props.children → Component এর ভিতরের content
│   │   ├── Default Props
```

```
│   │   ├── Prop Drilling সমস্যা
│   │   └── PropTypes (type checking)
│   │
│   ├── File Structure
│   │   ├── Feature-based structure (recommended)
│   │   │   └── src/features/auth/LoginForm.jsx
│   │   └── Type-based structure
│   │       └── src/components/, src/pages/, src/hooks/
│   └── Import/Export Components
│
├── React Hooks (সবচেয়ে গুরুত্বপূর্ণ)
│   ├── useState
│   │   ├── Component এর State (ডাটা) ম্যানেজ করা
│   │   ├── const [count, setCount] = useState(0);
│   │   ├── State পরিবর্তন হলে Component Re-render হয়
│   │   ├── Functional Update → setCount(prev => prev + 1)
│   │   ├── Object ও Array State
│   │   └── State কখনো সরাসরি পরিবর্তন করবেন না (Immutability)
│   │
│   ├── useEffect (অত্যন্ত গুরুত্বপূর্ণ)
│   │   ├── Side Effects handle করা (API call, DOM manipulation, timers)
│   │   ├── useEffect(() => { }, []) → Component Mount হলে একবার চলে
│   │   ├── useEffect(() => { }, [dependency]) → dependency পরিবর্তন হলে চলে
│   │   ├── useEffect(() => { }) → প্রতিবার render এ চলে (সাবধান!)
│   │   ├── Cleanup Function → return () => { clearInterval(timer); }
│   │   │   └── Component Unmount হলে বা পরবর্তী effect এর আগে চলে
│   │   └── Rules of Hooks
│   │       ├── শুধুমাত্র Top Level এ ব্যবহার করুন
│   │       └── শুধুমাত্র React Function Component বা Custom Hook এ
│   │
│   ├── useRef
│   │   ├── DOM Element এ Direct Access
│   │   ├── const inputRef = useRef(null);
│   │   ├── <input ref={inputRef} />
│   │   ├── inputRef.current.focus()
│   │   ├── Re-render ছাড়া Value সংরক্ষণ
│   │   └── Previous State/Props ধরে রাখা
│   │
│   ├── useContext (State Management)
│   │   ├── Prop Drilling সমস্যার সমাধান
│   │   ├── createContext() → Context তৈরি
│   │   ├── <Context.Provider value={data}> → ডাটা সরবরাহ
│   │   └── useContext(Context) → ডাটা ব্যবহার
│   │
```

```
│   ├── useReducer
│   │   ├── Complex State Logic এর জন্য
│   │   ├── Redux এর মতো Pattern
│   │   ├── const [state, dispatch] = useReducer(reducer, initialState);
│   │   ├── reducer function → (state, action) => newState
│   │   └── dispatch({ type: 'INCREMENT', payload: 1 })
│   │
│   ├── useMemo
│   │   ├── Expensive Calculation Cache করা
│   │   ├── const result = useMemo(() => heavyCalculation(data), [data]);
│   │   └── শুধুমাত্র dependency পরিবর্তন হলে Re-calculate হবে
│   │
│   ├── useCallback
│   │   ├── Function Memoization
│   │   ├── const handleClick = useCallback(() => { }, [dependency]);
│   │   └── Child Component এ Function pass করলে অপ্রয়োজনীয়
│   │       Re-render আটকানো
│   │
│   ├── useId
│   │   └── Unique ID তৈরি (Form elements এর জন্য)
│   │
│   ├── useTransition ও useDeferredValue
│   │   └── UI responsive রাখা (React 18)
│   │
│   └── Custom Hooks
│       ├── নিজের Hook তৈরি করা (use দিয়ে শুরু)
│       ├── উদাহরণ: useFetch, useLocalStorage, useDebounce
│       ├── Logic Reuse করা
│       └── Separation of Concerns
│
├── Event Handling in React
│   ├── onClick, onChange, onSubmit, onFocus, onBlur
│   ├── Event Object (e)
│   ├── e.preventDefault() → Form default behavior বন্ধ
│   ├── e.target.value → Input এর value
│   └── Synthetic Events
│
├── Forms in React
│   ├── Controlled Components
│   │   ├── State দিয়ে input control করা
│   │   ├── value={state} onChange={(e) => setState(e.target.value)}
│   │   └── React এর recommended approach
│   ├── Uncontrolled Components
│   │   └── useRef দিয়ে input access করা
```

```
│   ├── Form Submission
│   ├── Form Validation
│   │   ├── Manual Validation
│   │   └── React Hook Form (Library - অবশ্যই শিখুন)
│   │       ├── useForm()
│   │       ├── register, handleSubmit, errors
│   │       ├── Yup/Zod Integration (Schema Validation)
│   │       └── Performance অনেক ভালো
│   └── File Upload Handling
│
└── Styling in React
    ├── CSS Modules → styles.module.css
    ├── Tailwind CSS (সবচেয়ে জনপ্রিয়)
    ├── Styled Components
    ├── Inline Styles
    ├── CSS-in-JS (Emotion)
    └── Component Libraries
        ├── Shadcn UI (Recommended)
        ├── Material UI (MUI)
        ├── Ant Design
        ├── Chakra UI
        └── DaisyUI (Tailwind based)
```

ব্যাখ্যা:
React হলো Component-based Library। পুরো UI কে ছোট ছোট পুনরায় ব্যবহারযোগ্য Component এ ভাগ করা হয়। useState দিয়ে component এর ডাটা (State) রাখা হয় - state পরিবর্তন হলে component আবার render হয়। useEffect দিয়ে API call, timer ইত্যাদি side effect handle করা হয়। Dependency array খালি [] দিলে শুধু mount এ চলে, dependency দিলে সেটা পরিবর্তন হলে চলে। Props হলো Parent থেকে Child এ ডাটা পাঠানোর মাধ্যম এবং এটি পরিবর্তন করা যায় না। Context API দিয়ে prop drilling ছাড়া যেকোনো component এ ডাটা পাঠানো যায়।

2.2 React Router
text
বিষয়সমূহ:

```
├── React Router DOM (v6)
│   ├── Installation → npm install react-router-dom
│   ├── BrowserRouter → পুরো app wrap করা
│   ├── Routes ও Route
│   │   ├── <Route path="/" element={<Home />} />
│   │   ├── <Route path="/about" element={<About />} />
│   │   └── <Route path="*" element={<NotFound />} /> → 404 page
│   │
│   ├── Navigation
│   │   ├── <Link to="/about">About</Link> → SPA navigation
│   │   ├── <NavLink> → Active link styling
```

```
│   │   └── useNavigate() → Programmatic navigation
│   │       └── const navigate = useNavigate(); navigate('/dashboard');
│   │
│   ├── Dynamic Routes
│   │   ├── <Route path="/user/:id" element={<User />} />
│   │   └── useParams() → const { id } = useParams();
│   │
│   ├── Nested Routes
│   │   ├── <Route path="/dashboard" element={<Dashboard />}>
│   │   │   <Route index element={<DashboardHome />} />
│   │   │   <Route path="settings" element={<Settings />} />
│   │   └── </Route>
│   │   └── <Outlet /> → Nested content render করার জায়গা
│   │
│   ├── Query Parameters
│   │   └── useSearchParams()
│   │
│   ├── Protected Routes
│   │   └── Authentication check করে route access দেওয়া
│   │
│   ├── Lazy Loading Routes
│   │   ├── React.lazy(() => import('./pages/About'))
│   │   └── <Suspense fallback={<Loading />}>
│   │
│   ├── useLocation()
│   └── Loader ও Action (v6.4+)
│
└── Layout Routes
    └── Common Header/Footer/Sidebar সব page এ দেখানো
```

2.3 State Management

text

```
বিষয়সমূহ:
├── Local State → useState (single component)
├── Lifted State → Parent component এ state রেখে props দিয়ে share
├── Context API → Global state (ছোট-মাঝারি app)
│
├── Redux Toolkit (RTK) - বড় app এর জন্য (অবশ্যই শিখুন)
│   ├── Redux কি → Predictable State Container
│   ├── Core Concepts
│   │   ├── Store → সব state একটি জায়গায়
│   │   ├── Slice → state + reducers একসাথে
│   │   ├── Action → কি পরিবর্তন হবে
│   │   ├── Reducer → কিভাবে পরিবর্তন হবে
```

```
│   │   └── Dispatch → Action পাঠানো
│   ├── configureStore()
│   ├── createSlice()
│   ├── useSelector() → Store থেকে state পড়া
│   ├── useDispatch() → Action dispatch করা
│   ├── Redux DevTools
│   ├── RTK Query (API Calling)
│   │   ├── createApi()
│   │   ├── fetchBaseQuery()
│   │   ├── Auto-generated hooks
│   │   ├── Caching
│   │   └── Automatic re-fetching
│   └── createAsyncThunk (Async Operations)
│
├── Zustand (সহজ বিকল্প - জনপ্রিয় হচ্ছে)
│   ├── Minimal boilerplate
│   ├── const useStore = create((set) => ({ count: 0, inc: () => set(state => ({ count: state.count + 1
})) }))
│   └── No Provider needed
│
├── TanStack Query / React Query (Server State - অবশ্যই শিখুন)
│   ├── Server State ও Client State আলাদা
│   ├── useQuery() → Data Fetching
│   ├── useMutation() → Data Mutation (POST, PUT, DELETE)
│   ├── Caching → একই data বারবার fetch না করা
│   ├── Background Refetching
│   ├── Pagination ও Infinite Scroll
│   ├── Optimistic Updates
│   ├── Query Invalidation
│   └── Error ও Loading states
│
└── Jotai / Recoil (ঐচ্ছিক)
```

ব্যাখ্যা:
State Management মানে অ্যাপের ডাটা সংগঠিতভাবে রাখা ও পরিচালনা করা। ছোট অ্যাপে useState ও Context API যথেষ্ট। বড় অ্যাপে Redux Toolkit ব্যবহার করুন - এটি একটি Central Store এ সব ডাটা রাখে, createSlice দিয়ে state ও reducers একসাথে লেখা যায়। TanStack Query Server State (API থেকে আসা ডাটা) handle করার জন্য সেরা - এটি caching, auto-refetching, loading/error states সব আপনা আপনি handle করে। RTK Query ও একই কাজ করে তবে Redux ecosystem এর সাথে integrated।

2.4 API Integration
text
বিষয়সমূহ:

```
├── Fetch API (Built-in)
│   ├── GET → const res = await fetch(url);
```

```
│   ├── POST → fetch(url, { method: 'POST', headers: {...}, body: JSON.stringify(data) })
│   ├── PUT/PATCH
│   ├── DELETE
│   └── Error Handling
│
├── Axios (Popular Library - শিখুন)
│   ├── npm install axios
│   ├── axios.get(url)
│   ├── axios.post(url, data)
│   ├── axios.put(url, data)
│   ├── axios.delete(url)
│   ├── Axios Instance (Base URL, Headers)
│   ├── Interceptors (Request ও Response)
│   │   └── Token automatically যোগ করা
│   └── Error Handling
│
├── API Integration Patterns
│   ├── Loading, Error, Success states handle
│   ├── Custom Hooks for API (useFetch)
│   ├── Abort Controller (Request Cancel)
│   └── Environment Variables (.env)
│
└── Real-time Communication
    ├── WebSocket
    └── Socket.io (Real-time features)
```

2.5 Additional React Topics

text

```
বিষয়সমূহ:
├── React Performance Optimization
│   ├── React.memo() → Unnecessary re-render আটকানো
│   ├── useMemo() ও useCallback()
│   ├── Code Splitting (React.lazy + Suspense)
│   ├── Virtualization (react-window, react-virtualized)
│   ├── Image Optimization
│   ├── Bundle Analysis
│   └── React DevTools Profiler
│
├── Testing (গুরুত্বপূর্ণ)
│   ├── Jest (Test Runner)
│   ├── React Testing Library
│   ├── Unit Testing
│   ├── Integration Testing
│   ├── Component Testing
```

```
│   ├── Mocking
│   └── Test Coverage
│
├── TypeScript with React (অবশ্যই শিখুন)
│   ├── TypeScript কি → JavaScript + Types
│   ├── Basic Types (string, number, boolean, array, object)
│   ├── Interface ও Type
│   ├── Generics
│   ├── Enum
│   ├── Union ও Intersection Types
│   ├── React + TypeScript
│   │   ├── Component Props typing
│   │   ├── useState typing
│   │   ├── Event typing
│   │   ├── useRef typing
│   │   └── API Response typing
│   └── কেন TypeScript → Bugs কমায়, IDE support ভালো, বড় project এ সহায়ক
│
├── Next.js (React Framework - অত্যন্ত গুরুত্বপূর্ণ)
│   ├── কেন Next.js → SSR, SSG, SEO, File-based Routing
│   ├── App Router (নতুন - এটা শিখুন)
│   ├── Pages ও Layouts
│   ├── Server Components vs Client Components
│   ├── Data Fetching
│   ├── API Routes
│   ├── Middleware
│   ├── Image Optimization (next/image)
│   ├── Font Optimization
│   ├── Metadata ও SEO
│   ├── Static ও Dynamic Rendering
│   └── Server Actions
│
├── Important Libraries
│   ├── React Hook Form → Form handling
│   ├── Zod → Schema validation
│   ├── Framer Motion → Animation
│   ├── React Icons
│   ├── React Toastify / Sonner → Notifications
│   ├── React DnD / dnd-kit → Drag and Drop
│   ├── Recharts / Chart.js → Charts
│   ├── date-fns / dayjs → Date manipulation
│   ├── React PDF
│   └── React Table / TanStack Table
│
```

```
└── Authentication (Frontend)
    ├── JWT Token Storage
    ├── Protected Routes
    ├── Login/Register Forms
    ├── Auth Context/State
    ├── Token Refresh
    └── OAuth (Google, GitHub Login)
```

🟠 Phase 3: Backend Development (Node.js & Express.js) — সময়: ৬-৮ সপ্তাহ

3.1 Node.js
text
বিষয়সমূহ:

```
├── Node.js পরিচিতি
│   ├── Node.js কি → Browser এর বাইরে JavaScript চালানোর Runtime
│   ├── V8 Engine
│   ├── Node.js কেন ব্যবহার করা হয়
│   ├── Node.js vs Browser JavaScript
│   │   └── Browser: DOM, Window | Node: fs, http, os, path
│   ├── Non-blocking I/O
│   ├── Event-Driven Architecture
│   └── npm (Node Package Manager)
│
├── Node.js Basics
│   ├── Node.js Installation
│   ├── REPL (Read-Eval-Print-Loop)
│   ├── Running JavaScript Files → node app.js
│   ├── Global Objects (global, __dirname, __filename, process)
│   ├── process.env → Environment Variables
│   └── Command Line Arguments (process.argv)
│
├── Node.js Modules
│   ├── Core/Built-in Modules
│   │   ├── fs (File System)
│   │   │   ├── fs.readFile() / fs.readFileSync()
│   │   │   ├── fs.writeFile() / fs.writeFileSync()
│   │   │   ├── fs.appendFile()
│   │   │   ├── fs.unlink() → ফাইল delete
│   │   │   ├── fs.mkdir() → ফোল্ডার তৈরি
│   │   │   └── fs.readdir() → ফোল্ডারের ফাইল তালিকা
│   │   │
│   │   ├── path
│   │   │   ├── path.join() → পাথ যোগ
│   │   │   ├── path.resolve() → absolute path
```

```
│   │   │   ├── path.extname() → extension
│   │   │   └── path.basename() → ফাইলের নাম
│   │   │
│   │   ├── http
│   │   │   ├── http.createServer()
│   │   │   ├── Request ও Response object
│   │   │   └── server.listen(port)
│   │   │
│   │   ├── os → Operating System info
│   │   ├── events → EventEmitter
│   │   ├── crypto → Hashing, Encryption
│   │   ├── url → URL parsing
│   │   ├── util → Utility functions
│   │   └── stream → Data streaming
│   │
│   ├── CommonJS Modules
│   │   ├── module.exports = { }
│   │   └── const module = require('./module')
│   │
│   └── ES Modules (package.json → "type": "module")
│       ├── export default / export { }
│       └── import module from './module.js'
│
├── npm (Node Package Manager)
│   ├── package.json → Project configuration
│   ├── npm init -y → package.json তৈরি
│   ├── npm install package → dependency install
│   ├── npm install -D package → devDependency
│   ├── npm install -g package → Global install
│   ├── npm uninstall package
│   ├── npm update
│   ├── npm scripts
│   │   └── "start": "node app.js", "dev": "nodemon app.js"
│   ├── package-lock.json → exact versions
│   ├── node_modules ফোল্ডার
│   ├── Semantic Versioning (^1.2.3, ~1.2.3)
│   ├── npx → package না install করে চালানো
│   └── npm vs yarn vs pnpm
│
├── Streams ও Buffers
│   ├── Buffer → Binary data handle
│   ├── Readable Streams
│   ├── Writable Streams
│   ├── Duplex Streams
```

```
│   ├── Transform Streams
│   └── Piping
│
├── Events
│   ├── EventEmitter class
│   ├── .on() → event listen
│   ├── .emit() → event trigger
│   ├── .once() → একবার listen
│   └── .removeListener()
│
└── Advanced Node.js
    ├── Cluster Module → Multiple processes
    ├── Worker Threads
    ├── Child Processes
    ├── Libuv ও Event Loop in Node.js
    └── Memory Management
```

ব্যাখ্যা:
Node.js দিয়ে JavaScript কে Server-side এ চালানো যায়। এটি V8 Engine (Chrome এর JavaScript Engine) ব্যবহার করে। Node.js Non-blocking মানে একটি কাজ শেষ হওয়ার জন্য অপেক্ষা না করে পরের কাজ শুরু করে। npm দিয়ে অন্যদের তৈরি package install করে ব্যবহার করা যায়। fs module দিয়ে ফাইল পড়া-লেখা, http module দিয়ে ওয়েব সার্ভার তৈরি, path module দিয়ে ফাইল পাথ handle করা হয়।

3.2 Express.js
text
বিষয়সমূহ:

```
├── Express.js পরিচিতি
│   ├── Express কি → Node.js এর জনপ্রিয় Web Framework
│   ├── কেন Express → Routing, Middleware, সহজ API তৈরি
│   ├── Installation → npm install express
│   └── Basic Server
│       ├── const express = require('express');
│       ├── const app = express();
│       └── app.listen(3000, () => console.log('Server running'));
│
├── Routing
│   ├── HTTP Methods
│   │   ├── app.get('/api/users', handler) → ডাটা পড়া
│   │   ├── app.post('/api/users', handler) → নতুন ডাটা তৈরি
│   │   ├── app.put('/api/users/:id', handler) → পুরো ডাটা আপডেট
│   │   ├── app.patch('/api/users/:id', handler) → আংশিক আপডেট
│   │   └── app.delete('/api/users/:id', handler) → ডাটা মুছা
│   │
│   ├── Route Parameters
│   │   ├── /api/users/:id → req.params.id
```

```
│   │   └── /api/users/:userId/posts/:postId
│   │
│   ├── Query Parameters
│   │   ├── /api/users?page=1&limit=10 → req.query.page
│   │   └── Searching, Filtering, Sorting, Pagination
│   │
│   ├── Request Body
│   │   ├── req.body → POST/PUT request এর data
│   │   └── app.use(express.json()) → Body parsing
│   │
│   ├── Response Methods
│   │   ├── res.send() → General response
│   │   ├── res.json() → JSON response
│   │   ├── res.status(200).json() → Status code + JSON
│   │   ├── res.redirect()
│   │   ├── res.render() → Template rendering
│   │   └── res.sendFile() → Static file
│   │
│   ├── Router (Modular Routing)
│   │   ├── const router = express.Router();
│   │   ├── router.get('/', handler);
│   │   ├── app.use('/api/users', userRouter);
│   │   └── আলাদা ফাইলে Routes রাখা
│   │
│   └── Route Chaining
│       └── router.route('/').get(getAll).post(create)
│
├── Middleware (অত্যন্ত গুরুত্বপূর্ণ)
│   ├── Middleware কি
│   │   └── Request ও Response এর মাঝে যে ফাংশন কাজ করে
│   │   └── (req, res, next) => { /* কাজ */ next(); }
│   │
│   ├── Types of Middleware
│   │   ├── Application-level → app.use(middleware)
│   │   ├── Router-level → router.use(middleware)
│   │   ├── Error-handling → (err, req, res, next)
│   │   ├── Built-in
│   │   │   ├── express.json() → JSON body parse
│   │   │   ├── express.urlencoded() → Form data parse
│   │   │   └── express.static() → Static files serve
│   │   └── Third-party
│   │       ├── cors → Cross-Origin Resource Sharing
│   │       ├── helmet → Security headers
│   │       ├── morgan → Request logging
```

```
│   │       ├── compression → Response compression
│   │       ├── cookie-parser → Cookie parse
│   │       ├── express-rate-limit → Rate limiting
│   │       ├── express-validator → Input validation
│   │       └── multer → File upload
│   │
│   ├── Custom Middleware
│   │   ├── Logger Middleware
│   │   ├── Auth Middleware
│   │   └── Error Handler Middleware
│   │
│   └── Middleware Execution Order
│       └── উপর থেকে নিচে ক্রমানুসারে চলে
│
├── Error Handling
│   ├── Synchronous Errors → try-catch
│   ├── Asynchronous Errors → next(error)
│   ├── Global Error Handler Middleware
│   ├── Custom Error Class (AppError)
│   ├── 404 Not Found Handler
│   ├── Operational vs Programming Errors
│   └── express-async-handler / express-async-errors
│
├── File Upload
│   ├── Multer
│   │   ├── Single File Upload
│   │   ├── Multiple File Upload
│   │   ├── File Filter (type validation)
│   │   ├── Size Limit
│   │   └── Storage (disk / memory)
│   └── Cloudinary Integration (Cloud storage)
│
├── Template Engines (ঐচ্ছিক)
│   ├── EJS
│   ├── Pug
│   └── Handlebars
│
└── Project Structure (MVC Pattern)
    ├── config/
    │   ├── db.js → Database connection
    │   └── config.js → Configuration
    ├── controllers/
    │   └── userController.js → Business logic
    ├── models/
```

```
│   └── userModel.js → Database schema
├── routes/
│   └── userRoutes.js → Route definitions
├── middleware/
│   ├── auth.js → Authentication
│   ├── errorHandler.js → Error handling
│   └── validate.js → Validation
├── utils/
│   ├── apiError.js → Custom Error
│   ├── apiResponse.js → Standard Response
│   └── helpers.js → Helper functions
├── validators/
│   └── userValidator.js → Input validation
├── app.js → Express app setup
├── server.js → Server start
├── .env → Environment variables
└── package.json
```

ব্যাখ্যা:
Express.js দিয়ে REST API তৈরি করা হয়। REST API মানে HTTP Methods (GET, POST, PUT, DELETE) ব্যবহার করে CRUD (Create, Read, Update, Delete) অপারেশন করা। Middleware হলো Express এর হৃদয় - প্রতিটি Request আসলে এটি বিভিন্ন Middleware এর মধ্য দিয়ে যায়, প্রতিটি Middleware কিছু কাজ করে তারপর next() দিয়ে পরেরটায় পাঠায়। MVC Pattern অনুযায়ী কোড সাজালে কোড clean ও maintainable হয় - Model (Database), View (Frontend), Controller (Business Logic)।

3.3 Authentication & Authorization
text
বিষয়সমূহ:

```
├── Authentication vs Authorization
│   ├── Authentication → কে (Who) → Login
│   └── Authorization → কি করতে পারবে (What) → Permission
│
├── Password Hashing
│   ├── bcrypt / bcryptjs
│   ├── Salt Rounds
│   ├── bcrypt.hash(password, 10) → Hash করা
│   └── bcrypt.compare(password, hashedPassword) → তুলনা
│
├── JWT (JSON Web Token) - সবচেয়ে গুরুত্বপূর্ণ
│   ├── JWT কি → User verify করার Token
│   ├── JWT Structure → Header.Payload.Signature
│   ├── jwt.sign(payload, secret, { expiresIn: '7d' }) → Token তৈরি
│   ├── jwt.verify(token, secret) → Token যাচাই
│   ├── Access Token (Short-lived: 15min - 1hr)
│   ├── Refresh Token (Long-lived: 7-30 days)
```

```
│   ├── Token Storage
│   │   ├── HTTP-Only Cookies (Recommended - XSS safe)
│   │   ├── Local Storage (XSS vulnerable)
│   │   └── Memory (Session-only)
│   ├── Token Refresh Strategy
│   └── Logout → Token invalidation
│
├── Session-based Authentication (ঐচ্ছিক)
│   ├── express-session
│   ├── Session Store (Memory, Redis, Database)
│   └── Cookie-based sessions
│
├── OAuth 2.0 ও Social Login
│   ├── OAuth কি → Third-party login
│   ├── Passport.js
│   │   ├── passport-local → Email/Password
│   │   ├── passport-jwt → JWT
│   │   ├── passport-google-oauth20 → Google Login
│   │   └── passport-github2 → GitHub Login
│   ├── Google OAuth Implementation
│   ├── GitHub OAuth Implementation
│   └── OAuth Flow বোঝা
│
├── Role-Based Access Control (RBAC)
│   ├── User Roles (admin, user, moderator)
│   ├── Permission Middleware
│   └── Route Protection
│
├── Security Best Practices
│   ├── Input Validation ও Sanitization
│   ├── SQL/NoSQL Injection Prevention
│   ├── XSS (Cross-Site Scripting) Prevention
│   ├── CSRF (Cross-Site Request Forgery) Prevention
│   ├── Rate Limiting
│   ├── CORS Configuration
│   ├── Helmet.js (Security Headers)
│   ├── HTTPS
│   ├── Environment Variables (.env)
│   └── Password Reset Flow (Email Verification)
│
└── Email Services
    ├── Nodemailer
    ├── SendGrid
    ├── Email Templates
```

```
    ├── Email Verification
    └── Password Reset Email
```

ব্যাখ্যা:
Authentication মানে ইউজারকে চিনতে পারা (Login), Authorization মানে ইউজার কি কি করতে পারবে (Permission)। Password কখনো সরাসরি Database এ রাখবেন না - bcrypt দিয়ে Hash করুন। JWT হলো একটি Token যা ইউজারের পরিচয় বহন করে - Login করলে server একটি JWT দেয়, পরবর্তী সব request এ এই token পাঠাতে হয়। HTTP-Only Cookie তে token রাখলে JavaScript থেকে access করা যায় না, তাই XSS attack থেকে সুরক্ষিত।

3.4 REST API Design
text
বিষয়সমূহ:

```
├── RESTful API Principles
│   ├── REST কি → Representational State Transfer
│   ├── Stateless → Server কোনো client state মনে রাখে না
│   ├── Client-Server Architecture
│   ├── Resource-based URLs
│   │   ├── ✅ /api/v1/users → Users collection
│   │   ├── ✅ /api/v1/users/:id → Single user
│   │   ├── ✅ /api/v1/users/:id/posts → User's posts
│   │   ├── ❌ /api/v1/getUsers
│   │   └── ❌ /api/v1/deleteUser
│   └── HTTP Status Codes
│       ├── 200 → OK
│       ├── 201 → Created
│       ├── 204 → No Content
│       ├── 400 → Bad Request
│       ├── 401 → Unauthorized
│       ├── 403 → Forbidden
│       ├── 404 → Not Found
│       ├── 409 → Conflict
│       ├── 422 → Unprocessable Entity
│       ├── 429 → Too Many Requests
│       └── 500 → Internal Server Error
│
├── API Versioning
│   └── /api/v1/users, /api/v2/users
│
├── Pagination
│   ├── Page-based → ?page=1&limit=10
│   └── Cursor-based → ?cursor=abc123&limit=10
│
├── Filtering, Sorting, Searching
│   ├── ?status=active&role=admin (Filter)
```

```
│   ├── ?sort=createdAt&order=desc (Sort)
│   └── ?search=john (Search)
│
├── API Response Format
│   └── {
│       "success": true,
│       "message": "Users fetched successfully",
│       "data": { "users": [...], "pagination": {...} }
│   }
│
├── Input Validation
│   ├── express-validator
│   ├── Joi
│   └── Zod
│
├── API Documentation
│   ├── Swagger / OpenAPI
│   ├── Postman Collection
│   └── API Blueprint
│
└── API Testing
    ├── Postman
    ├── Thunder Client (VS Code Extension)
    └── Insomnia
```

🟣 Phase 4: Database (MongoDB) — সময়: ৪-৬ সপ্তাহ

4.1 MongoDB
text
বিষয়সমূহ:

```
├── MongoDB পরিচিতি
│   ├── MongoDB কি → NoSQL Document Database
│   ├── SQL vs NoSQL
│   │   ├── SQL: Table, Row, Column → MySQL, PostgreSQL
│   │   └── NoSQL: Collection, Document, Field → MongoDB
│   ├── কখন MongoDB ব্যবহার করবেন
│   │   ├── Flexible schema দরকার হলে
│   │   ├── JSON-like data
│   │   ├── Horizontal scaling
│   │   └── Rapid development
│   ├── BSON (Binary JSON)
│   ├── MongoDB Atlas (Cloud Database - ফ্রি)
│   │   └── atlas.mongodb.com এ account করুন
│   └── MongoDB Compass (GUI Tool)
```

```
│
├── MongoDB CRUD Operations
│   ├── Database ও Collection
│   │   ├── use myDatabase → Database select/create
│   │   ├── db.createCollection('users')
│   │   └── show dbs, show collections
│   │
│   ├── Create
│   │   ├── db.users.insertOne({ name: "John", age: 25 })
│   │   └── db.users.insertMany([{...}, {...}])
│   │
│   ├── Read
│   │   ├── db.users.find() → সব documents
│   │   ├── db.users.find({ age: 25 }) → ফিল্টার
│   │   ├── db.users.findOne({ _id: ObjectId("...") }) → একটি
│   │   ├── Projection → db.users.find({}, { name: 1, age: 1 })
│   │   ├── Comparison: $eq, $ne, $gt, $gte, $lt, $lte, $in, $nin
│   │   ├── Logical: $and, $or, $not, $nor
│   │   ├── Element: $exists, $type
│   │   ├── Array: $all, $elemMatch, $size
│   │   ├── sort() → db.users.find().sort({ age: -1 })
│   │   ├── limit() → db.users.find().limit(10)
│   │   ├── skip() → db.users.find().skip(10) (Pagination)
│   │   └── count() / countDocuments()
│   │
│   ├── Update
│   │   ├── db.users.updateOne({ _id: id }, { $set: { age: 26 } })
│   │   ├── db.users.updateMany({ status: "active" }, { $set: {...} })
│   │   ├── Update Operators
│   │   │   ├── $set → মান সেট
│   │   │   ├── $unset → field মুছা
│   │   │   ├── $inc → বাড়ানো/কমানো
│   │   │   ├── $push → Array তে যোগ
│   │   │   ├── $pull → Array থেকে বাদ
│   │   │   ├── $addToSet → Unique যোগ
│   │   │   ├── $rename → Field নাম পরিবর্তন
│   │   │   └── $min, $max, $mul
│   │   └── db.users.replaceOne()
│   │
│   └── Delete
│       ├── db.users.deleteOne({ _id: id })
│       └── db.users.deleteMany({ status: "inactive" })
│
├── Indexing
```

```
│   ├── Index কি → ডাটা দ্রুত খোঁজার জন্য
│   ├── db.users.createIndex({ email: 1 }) → Ascending
│   ├── Unique Index → { unique: true }
│   ├── Compound Index → { name: 1, age: -1 }
│   ├── Text Index → Full-text search
│   ├── TTL Index → Auto-delete after time
│   └── explain() → Query performance দেখা
│
├── Aggregation Pipeline
│   ├── $match → Filter
│   ├── $group → Group by
│   ├── $sort → Sort
│   ├── $project → Field selection
│   ├── $limit, $skip
│   ├── $lookup → Join (another collection)
│   ├── $unwind → Array flatten
│   ├── $count → Count
│   ├── $addFields → New fields
│   └── Pipeline Example:
│       db.orders.aggregate([
│           { $match: { status: "completed" } },
│           { $group: { _id: "$userId", total: { $sum: "$amount" } } },
│           { $sort: { total: -1 } },
│           { $limit: 10 }
│       ])
│
└── MongoDB Advanced
    ├── Transactions (Multi-document)
    ├── Replica Sets
    ├── Sharding
    ├── Change Streams
    ├── Schema Validation
    └── Geospatial Queries
```

4.2 Mongoose (MongoDB ODM)

text

```
বিষয়সমূহ:
├── Mongoose পরিচিতি
│   ├── Mongoose কি → MongoDB এর Object Document Mapper
│   ├── কেন Mongoose → Schema, Validation, Middleware, Population
│   └── Installation → npm install mongoose
│
├── Connection
│   ├── mongoose.connect(MONGODB_URI)
```

```
│   ├── Connection Events (connected, error, disconnected)
│   └── Connection Options
│
├── Schema ও Model
│   ├── Schema Definition
│   │   const userSchema = new mongoose.Schema({
│   │       name: {
│   │           type: String,
│   │           required: [true, 'Name is required'],
│   │           trim: true,
│   │           minlength: [2, 'Minimum 2 characters'],
│   │           maxlength: [50, 'Maximum 50 characters']
│   │       },
│   │       email: {
│   │           type: String,
│   │           required: true,
│   │           unique: true,
│   │           lowercase: true,
│   │           match: [/regex/, 'Invalid email']
│   │       },
│   │       age: {
│   │           type: Number,
│   │           min: [18, 'Minimum age 18'],
│   │           max: 100
│   │       },
│   │       role: {
│   │           type: String,
│   │           enum: ['user', 'admin', 'moderator'],
│   │           default: 'user'
│   │       },
│   │       isActive: { type: Boolean, default: true },
│   │       tags: [String], → Array of Strings
│   │       address: {
│   │           street: String,
│   │           city: String  → Nested Object
│   │       },
│   │       avatar: String,
│   │       password: { type: String, select: false } → Query তে আসবে না
│   │   }, {
│   │       timestamps: true → createdAt, updatedAt automatically
│   │   });
│   │
│   ├── Schema Types
│   │   └── String, Number, Boolean, Date, ObjectId, Array, Mixed, Map, Buffer
```

```
│   │
│   └── Model Creation
│       └── const User = mongoose.model('User', userSchema);
│
├── CRUD with Mongoose
│   ├── Create
│   │   ├── const user = await User.create(data);
│   │   ├── const user = new User(data); await user.save();
│   │   └── await User.insertMany([...]);
│   │
│   ├── Read
│   │   ├── const users = await User.find();
│   │   ├── const users = await User.find({ role: 'admin' });
│   │   ├── const user = await User.findById(id);
│   │   ├── const user = await User.findOne({ email });
│   │   ├── Query Chaining
│   │   │   └── User.find({ role: 'user' })
│   │   │       .select('name email')  → শুধু এই fields
│   │   │       .sort('-createdAt')    → নতুন আগে
│   │   │       .skip(10)             → প্রথম ১০টি বাদ
│   │   │       .limit(10)            → ১০টি নাও
│   │   │       .populate('posts')     → Reference populate
│   │   │       .lean()               → Plain JS Object (faster)
│   │   │       .exec();
│   │   │
│   │   ├── Comparison → find({ age: { $gte: 18, $lte: 65 } })
│   │   ├── Text Search → find({ $text: { $search: "keyword" } })
│   │   └── countDocuments()
│   │
│   ├── Update
│   │   ├── await User.findByIdAndUpdate(id, data, { new: true, runValidators: true });
│   │   ├── await User.findOneAndUpdate(filter, data, options);
│   │   ├── await User.updateOne(filter, { $set: data });
│   │   └── await User.updateMany(filter, data);
│   │
│   └── Delete
│       ├── await User.findByIdAndDelete(id);
│       ├── await User.findOneAndDelete(filter);
│       ├── await User.deleteOne(filter);
│       └── await User.deleteMany(filter);
│
├── Mongoose Middleware (Hooks)
│   ├── Pre Hooks → Schema.pre('save', function(next) { })
│   │   ├── Password hash before save
```

```
│   │   └── Data modification before save
│   ├── Post Hooks → Schema.post('save', function(doc) { })
│   │   └── Log after save
│   ├── Hook Types: save, validate, remove, find, findOne, updateOne
│   └── Document vs Query Middleware
│
├── Mongoose Virtuals
│   ├── Virtual Fields (Database এ save হয় না)
│   └── userSchema.virtual('fullName').get(function() {
│       return `${this.firstName} ${this.lastName}`;
│   });
│
├── Instance ও Static Methods
│   ├── Instance Methods → Individual document এ
│   │   └── userSchema.methods.comparePassword = async function(password) { }
│   └── Static Methods → Model level এ
│       └── userSchema.statics.findByEmail = function(email) { }
│
├── Population (References/Relations)
│   ├── ObjectId Reference
│   │   └── author: { type: mongoose.Schema.Types.ObjectId, ref: 'User' }
│   ├── .populate('author') → Reference এর data আনা
│   ├── .populate('author', 'name email') → নির্দিষ্ট fields
│   ├── Nested Population
│   └── Virtual Population
│
├── Validation
│   ├── Built-in Validators
│   │   └── required, min, max, minlength, maxlength, enum, match
│   ├── Custom Validators
│   │   └── validate: { validator: fn, message: 'Error' }
│   └── Validation Errors Handling
│
└── Advanced Mongoose
    ├── Transactions
    ├── Discriminators (Inheritance)
    ├── Plugins
    ├── Lean Queries
    ├── Pagination Plugin (mongoose-paginate-v2)
    └── Soft Delete
```

ব্যাখ্যা:

MongoDB হলো NoSQL Database যেখানে ডাটা JSON-like Document হিসেবে থাকে। SQL Database এ Table, Row, Column থাকে; MongoDB তে Collection, Document, Field থাকে। Mongoose হলো MongoDB এর ODM (Object Document Mapper) যা Schema (ডাটার গঠন), Validation (ডাটা যাচাই),

Middleware (কাজের আগে/পরে কিছু করা) ইত্যাদি সুবিধা দেয়। Population দিয়ে একটি Document থেকে অন্য Collection এর reference data আনা যায় (SQL এর JOIN এর মতো)। Aggregation Pipeline দিয়ে complex data analysis করা যায়।

🔴 Phase 5: Full Stack Integration — সময়: ৪-৬ সপ্তাহ

5.1 MERN Stack সংযোগ
text
বিষয়সমূহ:
├── Frontend-Backend সংযোগ
│ ├── CORS Setup
│ │ └── npm install cors → app.use(cors({ origin: 'http://localhost:5173', credentials: true }))
│ ├── Proxy Setup (Development)
│ │ └── Vite: vite.config.js → proxy
│ ├── API Base URL Management
│ │ └── .env → VITE_API_URL=http://localhost:5000/api/v1
│ ├── Axios Instance
│ │ └── const api = axios.create({ baseURL: import.meta.env.VITE_API_URL, withCredentials: true })
│ └── Error Handling (Frontend ↔ Backend)
│
├── Authentication Flow
│ ├── Registration Flow
│ │ ├── Frontend: Form → Validation → API call
│ │ ├── Backend: Validate → Check exists → Hash password → Save → Generate JWT → Response
│ │ └── Frontend: Save token → Redirect to dashboard
│ │
│ ├── Login Flow
│ │ ├── Frontend: Form → API call
│ │ ├── Backend: Find user → Compare password → Generate JWT → Set cookie
│ │ └── Frontend: Save user data → Redirect
│ │
│ ├── Protected Route Flow
│ │ ├── Frontend: Check auth state → Redirect if not logged in
│ │ ├── API call: Send token in header/cookie
│ │ └── Backend: Verify token → Allow/Deny
│ │
│ ├── Token Refresh Flow
│ └── Logout Flow
│
├── Full CRUD Application
│ ├── Project Planning
│ │ ├── Requirements Analysis

```
│   │   ├── Database Schema Design
│   │   ├── API Endpoint Design
│   │   ├── UI/UX Wireframe
│   │   └── Component Structure
│   │
│   ├── Backend Development
│   │   ├── Database Models
│   │   ├── API Routes & Controllers
│   │   ├── Middleware (Auth, Validation, Error)
│   │   ├── Testing with Postman
│   │   └── API Documentation
│   │
│   ├── Frontend Development
│   │   ├── Component Creation
│   │   ├── State Management Setup
│   │   ├── API Integration
│   │   ├── Form Handling & Validation
│   │   ├── Error & Loading States
│   │   ├── Responsive Design
│   │   └── User Experience (UX)
│   │
│   └── Integration
│       ├── Connect Frontend to Backend
│       ├── Test all CRUD operations
│       ├── Error Handling
│       └── Edge Cases
│
├── Image/File Upload (Full Stack)
│   ├── Frontend: File Input → FormData → API call
│   ├── Backend: Multer → Process → Save/Upload
│   ├── Cloudinary Integration
│   │   ├── Upload to Cloudinary
│   │   ├── Image Transformation
│   │   └── Delete from Cloudinary
│   └── Display Images in Frontend
│
├── Real-time Features
│   ├── Socket.io
│   │   ├── Server Setup → io.on('connection', socket => { })
│   │   ├── Client Setup → io('http://localhost:5000')
│   │   ├── Events: emit, on, broadcast
│   │   ├── Rooms & Namespaces
│   │   └── Use Cases
│   │       ├── Real-time Chat
```

```
│   │       ├── Notifications
│   │       ├── Live Updates
│   │       └── Online Users
│   └── Server-Sent Events (SSE)
│
├── Payment Integration
│   ├── Stripe
│   │   ├── Stripe Checkout
│   │   ├── Payment Intent
│   │   ├── Webhooks
│   │   └── Subscription
│   ├── SSLCommerz (বাংলাদেশ)
│   └── bKash/Nagad Integration
│
└── Email Integration
    ├── Nodemailer Setup
    ├── Email Verification
    ├── Password Reset
    ├── Order Confirmation
    └── Welcome Email
```

5.2 প্রজেক্ট আইডিয়া (Practice)

text

```
📝 Beginner Projects:
├── 1. Todo App (CRUD)
├── 2. Notes App
├── 3. Contact Manager
├── 4. Blog App (with Auth)
└── 5. Weather App

📝 Intermediate Projects:
├── 1. E-commerce Website
│   ├── Product Listing, Filtering, Search
│   ├── Shopping Cart
│   ├── User Auth
│   ├── Order Management
│   ├── Payment Integration
│   ├── Admin Dashboard
│   └── Reviews & Ratings
│
├── 2. Social Media App
│   ├── User Profile
│   ├── Posts (CRUD)
│   ├── Like, Comment, Share
```

```
│   ├── Follow/Unfollow
│   ├── News Feed
│   ├── Real-time Notifications
│   └── Image Upload
│
├── 3. Chat Application
│   ├── Real-time Messaging (Socket.io)
│   ├── Group Chat
│   ├── Online Status
│   ├── Message Read/Unread
│   ├── File/Image Sharing
│   └── Typing Indicator
│
├── 4. Job Portal
│   ├── Job Posting (Employer)
│   ├── Job Search & Filter (Candidate)
│   ├── Application Management
│   ├── Resume Upload
│   └── Email Notifications
│
└── 5. Learning Management System (LMS)
    ├── Course Creation (Admin/Instructor)
    ├── Video Lessons
    ├── Quiz & Assignments
    ├── Progress Tracking
    ├── Certificate Generation
    └── Payment for Courses
```

📝 Advanced Projects (Portfolio এ রাখুন):

```
├── 1. Project Management Tool (Trello Clone)
├── 2. Food Delivery App
├── 3. Booking System (Hotel/Appointment)
├── 4. Video Streaming Platform
└── 5. Multi-vendor Marketplace
```

🔴 Phase 6: Advanced Topics — সময়: ৪-৬ সপ্তাহ

6.1 Advanced Concepts

text

```
বিষয়সমূহ:
├── Caching
│   ├── Redis
│   │   ├── Redis কি → In-Memory Data Store
│   │   ├── Key-Value Store
```

```
│   │   ├── Caching Strategy
│   │   ├── Session Store
│   │   ├── Rate Limiting
│   │   └── Pub/Sub (Real-time)
│   ├── Node-cache (Simple caching)
│   └── HTTP Caching (Cache-Control headers)
│
├── Message Queues
│   ├── Bull / BullMQ (Node.js)
│   ├── RabbitMQ
│   └── Use Cases: Email sending, Image processing, Background jobs
│
├── Search
│   ├── MongoDB Text Search
│   ├── Elasticsearch (Advanced)
│   └── Algolia
│
├── Microservices (ঐচ্ছিক - বোনাস)
│   ├── Microservices vs Monolithic
│   ├── API Gateway
│   ├── Service Communication
│   └── Docker ও Kubernetes
│
├── GraphQL (ঐচ্ছিক - বোনাস)
│   ├── GraphQL vs REST
│   ├── Schema, Query, Mutation
│   ├── Apollo Server
│   └── Apollo Client
│
├── WebSockets
│   ├── Socket.io Deep Dive
│   ├── WebSocket Protocol
│   └── Real-time patterns
│
├── Task Scheduling
│   ├── node-cron
│   └── Agenda.js
│
├── Logging
│   ├── Winston
│   ├── Morgan
│   └── Log Levels (error, warn, info, debug)
│
├── Rate Limiting & Throttling
```

```
│   ├── express-rate-limit
│   ├── Sliding Window
│   └── Token Bucket
│
└── Webhook
    ├── Webhook কি → Event-driven HTTP callback
    ├── Payment Webhooks (Stripe)
    └── GitHub Webhooks
```

🟤 Phase 7: DevOps & Deployment — সময়: ২-৩ সপ্তাহ

7.1 Deployment
text
বিষয়সমূহ:

```
├── Environment Setup
│   ├── Environment Variables (.env)
│   ├── Development vs Production Config
│   └── dotenv package
│
├── Frontend Deployment
│   ├── Vercel (Recommended for React/Next.js)
│   │   ├── GitHub Integration
│   │   ├── Automatic Deployments
│   │   ├── Custom Domain
│   │   └── Environment Variables
│   ├── Netlify
│   ├── GitHub Pages
│   └── Firebase Hosting
│
├── Backend Deployment
│   ├── Render (Recommended - Free tier)
│   ├── Railway
│   ├── Vercel (Serverless Functions)
│   ├── DigitalOcean
│   ├── AWS (EC2, Lambda, Elastic Beanstalk)
│   ├── Google Cloud Platform
│   └── Heroku (Paid now)
│
├── Database Deployment
│   ├── MongoDB Atlas (Cloud - Recommended)
│   └── Database Backup Strategy
│
├── Docker (গুরুত্বপূর্ণ)
│   ├── Docker কি → Container Platform
```

```
│   ├── Dockerfile → Image build configuration
│   ├── docker-compose.yml → Multi-container setup
│   ├── Docker Commands
│   │   ├── docker build
│   │   ├── docker run
│   │   ├── docker-compose up
│   │   └── docker-compose down
│   ├── Docker Hub
│   └── Dockerizing MERN App
│       ├── Frontend Dockerfile
│       ├── Backend Dockerfile
│       └── docker-compose.yml (Frontend + Backend + MongoDB)
│
├── CI/CD (Continuous Integration / Continuous Deployment)
│   ├── GitHub Actions
│   │   ├── Workflow file (.github/workflows/deploy.yml)
│   │   ├── Automated Testing
│   │   ├── Automated Deployment
│   │   └── Environment Secrets
│   ├── GitLab CI/CD
│   └── Jenkins
│
├── Domain & SSL
│   ├── Custom Domain Setup
│   ├── DNS Configuration
│   ├── SSL Certificate (HTTPS)
│   │   └── Let's Encrypt (Free)
│   └── Cloudflare (CDN + Security)
│
├── Nginx (Reverse Proxy)
│   ├── Nginx কি
│   ├── Reverse Proxy Configuration
│   ├── Load Balancing
│   └── Static File Serving
│
├── Linux Basics (Server management)
│   ├── Basic Commands (ls, cd, mkdir, rm, cp, mv)
│   ├── File Permissions (chmod, chown)
│   ├── Process Management (ps, kill, top)
│   ├── SSH Connection
│   ├── Package Management (apt, yum)
│   └── Systemd Services
│
└── Monitoring & Performance
```

```
    ├── PM2 (Node.js Process Manager)
    │   ├── pm2 start app.js
    │   ├── pm2 list, pm2 logs
    │   ├── pm2 restart, pm2 stop
    │   └── Cluster Mode
    ├── Application Monitoring
    │   ├── New Relic
    │   ├── DataDog
    │   └── Sentry (Error Tracking)
    ├── Health Check Endpoint
    └── Performance Optimization
        ├── Compression
        ├── CDN
        ├── Database Indexing
        ├── Connection Pooling
        └── Memory Optimization
```

ব্যাখ্যা:
Deployment মানে আপনার অ্যাপ্লিকেশন ইন্টারনেটে সবার জন্য চালু করা। Frontend সাধারণত Vercel বা Netlify তে, Backend Render বা Railway তে, Database MongoDB Atlas এ deploy করা হয়। Docker দিয়ে অ্যাপ একটি container এ প্যাক করা যায় যা যেকোনো জায়গায় একইভাবে চলবে। CI/CD মানে কোড push করলে আপনা আপনি test হবে এবং deploy হবে। PM2 দিয়ে Node.js app production এ চালানো হয় - app crash হলে আপনা আপনি restart হয়।

🟡 Phase 8: Professional Skills — চলমান

8.1 চাকরি পাওয়ার জন্য
text
বিষয়সমূহ:

```
├── Portfolio Website
│   ├── নিজের ওয়েবসাইট তৈরি করুন
│   ├── About, Skills, Projects, Contact sections
│   ├── Responsive ও আকর্ষণীয়
│   └── Custom Domain
│
├── GitHub Profile
│   ├── Clean ও Organized Repositories
│   ├── README.md প্রতিটি প্রজেক্টে
│   ├── Contribution Graph (Green squares)
│   ├── Pinned Repositories
│   └── GitHub Profile README
│
├── Projects (কমপক্ষে ৩-৪টি ভালো প্রজেক্ট)
│   ├── প্রতিটি প্রজেক্টে
│   │   ├── Live Demo Link
```

```
│   │   ├── GitHub Repository
│   │   ├── README (Description, Tech Stack, Features, Setup)
│   │   ├── Screenshots/Demo Video
│   │   └── Clean Code
│   └── বৈচিত্র্যময় প্রজেক্ট (E-commerce, Chat, Dashboard, etc.)
│
├── Resume/CV
│   ├── ১ পৃষ্ঠা
│   ├── Skills, Projects, Education, Experience
│   ├── ATS-Friendly Format
│   └── PDF Format
│
├── LinkedIn Profile
│   ├── Professional Photo
│   ├── Headline: MERN Stack Developer
│   ├── About Section
│   ├── Skills & Endorsements
│   ├── Projects & Accomplishments
│   └── Networking
│
├── Interview Preparation
│   ├── JavaScript Interview Questions
│   │   ├── Closure, Hoisting, this
│   │   ├── Event Loop
│   │   ├── Promises, Async/Await
│   │   ├── Prototype, OOP
│   │   └── Array Methods
│   │
│   ├── React Interview Questions
│   │   ├── Virtual DOM
│   │   ├── Hooks (useState, useEffect, etc.)
│   │   ├── Component Lifecycle
│   │   ├── State Management
│   │   ├── Performance Optimization
│   │   └── React Router
│   │
│   ├── Node.js/Express Interview Questions
│   │   ├── Event Loop in Node
│   │   ├── Middleware
│   │   ├── Authentication (JWT)
│   │   ├── REST API Design
│   │   └── Error Handling
│   │
│   ├── MongoDB Interview Questions
```

```
│   │   ├── SQL vs NoSQL
│   │   ├── Aggregation
│   │   ├── Indexing
│   │   └── Mongoose
│   │
│   ├── System Design (Basic)
│   │   ├── URL Shortener Design
│   │   ├── Chat Application Design
│   │   └── E-commerce System Design
│   │
│   └── Data Structures & Algorithms (DSA)
│       ├── Arrays ও Strings
│       ├── Linked List
│       ├── Stack ও Queue
│       ├── Hash Table / Map
│       ├── Tree ও Binary Search Tree
│       ├── Sorting Algorithms
│       ├── Searching Algorithms
│       ├── Recursion
│       ├── Two Pointers
│       ├── Sliding Window
│       └── LeetCode/HackerRank Practice
│
├── Soft Skills
│   ├── Communication
│   ├── Problem Solving
│   ├── Teamwork
│   ├── Time Management
│   ├── Learning Ability
│   └── English Proficiency
│
├── Freelancing (ঐচ্ছিক)
│   ├── Fiverr
│   ├── Upwork
│   ├── TopTal
│   ├── Local Clients
│   └── Agency
│
└── Continuous Learning
    ├── Tech Blogs পড়া
    ├── YouTube Channels Follow
    ├── Twitter/X Tech Community
    ├── Dev.to, Medium, Hashnode এ লেখা
    ├── Open Source Contribution
```

```
    ├── Tech Meetups ও Communities
    └── নতুন Technology শেখা চালিয়ে যাওয়া
```

📊 সময়সীমা সারসংক্ষেপ

text

```
╔═══════════════════════════════════════════════════════════════════════════════
═══╗
║                    MERN STACK ROADMAP TIMELINE                    ║
║═══════════════════════════════════════════════════════════════════════════════
═══║
║                                                                           ║
║  Phase 1: Foundation (HTML, CSS, JS, Git)   →  ৪-৬ সপ্তাহ   ║
║  Phase 2: React.js + Ecosystem              →  ৮-১২ সপ্তাহ  ║
║  Phase 3: Node.js + Express.js              →  ৬-৮ সপ্তাহ   ║
║  Phase 4: MongoDB + Mongoose                →  ৪-৬ সপ্তাহ   ║
║  Phase 5: Full Stack Integration            →  ৪-৬ সপ্তাহ   ║
║  Phase 6: Advanced Topics                   →  ৪-৬ সপ্তাহ   ║
║  Phase 7: DevOps & Deployment               →  ২-৩ সপ্তাহ   ║
║  Phase 8: Professional Skills               →  চলমান        ║
║                                                                           ║
║  ═══════════════════════════════════════════════════                      ║
║  মোট আনুমানিক সময়: ৮-১২ মাস (প্রতিদিন ৪-৬ ঘণ্টা)        ║
║  ═══════════════════════════════════════════════════                      ║
║                                                                           ║
║  📌 প্রতিদিন কমপক্ষে ৪ ঘণ্টা Practice করুন                ║
║  📌 শুধু দেখবেন না, কোড লিখুন                             ║
║  📌 প্রতি Phase শেষে একটি প্রজেক্ট করুন                   ║
║  📌 প্রতিদিন GitHub এ Push করুন                            ║
║                                                                           ║
╚═══════════════════════════════════════════════════════════════════════════════
═══╝
```

🎯 গুরুত্বপূর্ণ টিপস

text

```
1️⃣ Tutorial Hell এড়িয়ে চলুন
   → ভিডিও দেখে দেখে শুধু follow করবেন না
   → নিজে নিজে প্রজেক্ট তৈরি করুন
   → Error আসলে Google/Stack Overflow এ খুঁজুন

2️⃣ প্রজেক্ট-ভিত্তিক শিখুন
   → প্রতিটি concept শেখার পর ছোট প্রজেক্টে apply করুন
   → ধীরে ধীরে বড় প্রজেক্ট করুন

3️⃣ ডকুমেন্টেশন পড়ার অভ্যাস করুন
```

→ React Docs, MDN, Node.js Docs, Express Docs, MongoDB Docs

4 কোড পড়ুন
→ GitHub এ অন্যদের কোড পড়ুন
→ Open Source project explore করুন

5 Community তে যুক্ত হোন
→ Facebook Groups, Discord Servers
→ Stack Overflow তে প্রশ্ন করুন ও উত্তর দিন

6 English ভালো করুন
→ Documentation, Error Messages সব English এ
→ Interview ও Communication এর জন্য জরুরি

7 নিয়মিত থাকুন
→ প্রতিদিন কিছু না কিছু শিখুন বা কোড করুন
→ ছোট ছোট ধাপে এগোন, হাল ছাড়বেন না